

W10D03 — Volume Mounting & Data Persistence

ML Engineer Journey | Phase 3 — Docker & Clean Architecture | 13 Mei 2026

1. Analogi: Buku Catatan vs Brankas Lab

Bayangkan kamu bekerja di sebuah lab kimia. Ada dua cara menyimpan catatan hasil eksperimen:

- Cara A** — kamu pakai buku catatan milikmu sendiri yang ada di meja. Semua orang di lab bisa langsung melihat, membuka, dan mengedit langsung dari mejamu. Kalau mejamu dipindah atau dibongkar, buku itu ikut hilang bersama meja.
- Cara B** — kamu titip catatan ke brankas pusat lab. Kamu tidak tahu persis di laci mana disimpan, tapi selama brankasnya ada, catatan itu selalu bisa diambil kembali. Brankas tidak peduli siapa yang pakai lab hari ini.

Di dunia Docker: **bind mount** = buku catatanmu di meja sendiri (path kamu tentukan). **Named volume** = brankas pusat Docker (Docker yang kelola lokasinya, kamu hanya kasih nama).

2. Kenapa Volume Diperlukan

Container bersifat **ephemeral** — dia bisa mati, restart, atau diganti imagenya kapan saja. Semua yang ditulis di dalam container filesystem akan hilang bersamanya. Volume adalah mekanisme Docker untuk memisahkan data dari lifecycle container, sehingga data tetap ada meskipun container dihapus dan dibuat ulang.

Tiga tipe storage di Docker:

- Bind Mount** — kamu mount direktori spesifik dari host ke dalam container. Perubahan langsung dua arah: host bisa baca file dari container, container bisa baca file dari host. Path ditentukan sepenuhnya oleh developer.
- Named Volume** — Docker membuat dan mengelola volume sendiri di `/var/lib/docker/volumes/`. Kamu hanya perlu referensikan nama. Data survive container restart, bahkan survive docker-compose down (tapi TIDAK survive docker-compose down -v).
- tmpfs** — disimpan di memory saja, tidak ke disk. Dipakai untuk data sensitif sementara. Tidak dipersist ke disk sama sekali.

3. Perbandingan Bind Mount vs Named Volume

Aspek	Bind Mount	Named Volume
Path ditentukan oleh	Kamu (developer)	Docker daemon
Lokasi di host	<code>./logs</code> , <code>/any/path</code>	<code>/var/lib/docker/volumes/...</code>
Cocok untuk	Dev, hot reload, log access	Production, database, state
Survive docker-compose down	Ya (file tetap di host)	Ya (volume tetap ada)
Survive docker-compose down -v	Tidak (tidak terpengaruh)	TIDAK — volume terhapus
Akses langsung dari host	Mudah, buka file explorer	Perlu perintah khusus
Portabilitas	Tidak — path bisa beda	Ya — Docker managed
Performance I/O	Sedikit lebih lambat	Lebih optimal

4. Implementasi di docker-compose.yml

Pola yang benar: bind mount untuk log (dev convenience), named volume untuk Redis data (production reliability). Deklarasi **volumes**: di level root wajib ada — tanpa ini Docker akan error saat mencoba mount named volume.

```
name: ml-risk-api          # project name explicit, tidak bergantung nama folder

services:
  api:
    build: .
    volumes:
      - ./logs:/app/logs    # BIND MOUNT: dev convenience, akses log langsung dari host
    depends_on:
      - redis
  redis:
    image: redis:7-alpine
    volumes:
      - redis-data:/data    # NAMED VOLUME: Docker managed, survive restart
    command: redis-server --appendonly yes    # AOF persistence
volumes:
  redis-data:              # WAJIB dideklarasikan di sini
```

Kenapa Redis pakai named volume, bukan bind mount? Karena kamu tidak perlu akses langsung ke file binary Redis dari host. Yang penting data survive restart. Named volume lebih clean dan portable. Log API sebaliknya — kamu ingin bisa *tail -f logs/app.log* langsung dari terminal host tanpa masuk ke container.

5. Perintah Volume yang Wajib Diketahui

Perintah	Efek
<code>docker volume ls</code>	List semua volume di sistem
<code>docker volume inspect <nama></code>	Detail volume: lokasi, labels, mountpoint
<code>docker volume rm <nama></code>	Hapus volume spesifik — DATA HILANG PERMANEN
<code>docker-compose down</code>	Stop + hapus container. Volume AMAN
<code>docker-compose down -v</code>	Stop + hapus container + HAPUS named volumes
<code>docker system prune --volumes</code>	Hapus semua resource tak terpakai termasuk volumes

6. Volume Name Prefix — Jebakan yang Sering Diabaikan

Docker Compose otomatis prefix nama volume dengan nama project — defaultnya diambil dari nama folder tempat docker-compose.yml berada. Jika folder bernama WEEK-8, maka volume yang terdaftar adalah **week-8_redis-data**, bukan **redis-data**.

Konsekuensi kritis: jika kamu rename folder atau pindah project ke direktori lain, Docker akan mencari volume dengan prefix baru yang belum ada. Container akan start dengan volume KOSONG — data lama tidak hilang secara teknis, tapi tidak ter-mount. Ini salah satu sumber kebingungan paling umum di production.

Solusi: Selalu definisikan project name secara eksplisit di docker-compose.yml:

```
name: ml-risk-api    # nama project explicit, tidak bergantung nama folder
```

Dengan ini, volume akan selalu bernama **ml-risk-api_redis-data** regardless of folder name, path, atau siapa yang menjalankan compose file tersebut.

7. Pola Verifikasi Persistensi

Konfigurasi volume yang benar harus dibuktikan, bukan diasumsikan. Pola verifikasi standar menggunakan dua mode eksekusi terpisah:

Langkah 1 — Tulis data test ke Redis:

```
python scripts/verify_persistence.py write
```

Langkah 2 — Restart container Redis:

```
docker-compose restart redis
```

Langkah 3 — Verifikasi data masih ada:

```
python scripts/verify_persistence.py read
```

Script yang baik tidak hanya mengecek apakah key ada, tapi juga memverifikasi value-nya tidak berubah atau corrupt (mismatch check). Gunakan **argparse** untuk CLI yang self-documenting, dan **single source of truth** (TEST_DATA dictionary) agar write dan read selalu menggunakan data yang identik — DRY Principle.

8. Hal Penting yang Harus Diingat

- **docker-compose down -v di production = bencana.** Named volume ikut terhapus. Bind mount aman karena Docker tidak punya wewenang atas host filesystem.
- **Named volume BISA dihapus** — via down -v, docker volume rm, atau docker system prune --volumes. Tidak ada yang 'aman secara default'.
- **ports di Redis sebaiknya dihapus di production.** Redis yang exposed ke host adalah unnecessary attack surface. Simpan di docker-compose.override.yml untuk dev saja.
- **AOF persistence (--appendonly yes)** memastikan setiap write ke Redis dicatat ke disk. Tanpa ini, data di dalam named volume pun bisa hilang jika container crash sebelum Redis sempat flush ke disk.
- **Backup volume:** docker run --rm -v redis-data:/data -v \$(pwd):/backup alpine tar czf /backup/backup.tar.gz -C /data . — pattern standar backup named volume ke file.